# TRAINING CHESS BOTS WITH VARIOUS MACHINE LEARNING TECHNIQUES

by

BHAVYA SINGH
URN: 6839571

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Dr Joey Sik Chun Lam

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Bhavya Singh
May 2026

# Abstract

Modern chess engines achieve superhuman play but depend on deep search trees that demand substantial computational resources. This dissertation asks whether a neural network can learn to play strong chess without any search at all. We train and compare three deep learning architectures, Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs), on millions of board positions labelled by Stockfish 16 at high depth. Each model receives an encoded board state and outputs a policy over legal moves, bypassing traditional tree search entirely.

We evaluate the resulting "search-less" models in two ways: first, by playing them against Stockfish at controlled depth limits to estimate their ELO ratings; second, by testing them on the Lichess Puzzle dataset to measure tactical accuracy across motifs such as forks, pins, and endgame conversions. Our comparison considers not only move accuracy but also inference speed and parameter efficiency, since a primary motivation for removing search is deployment on resource-constrained hardware.

The results show that each architecture captures different aspects of chess reasoning. CNNs excel at local pattern recognition, Transformers handle long-range piece coordination, and GNNs naturally represent the relational structure of legal moves. We discuss where each model fails, particularly in positions requiring deep calculation, and outline directions for closing the gap between search-based and search-free play.

# Acknowledgements

Write any personal words of thanks here. Typically, this space is used to thank your supervisor for their guidance, as well as anyone else who has supported the completion of this dissertation, for example by discussing results and their interpretation or reviewing write ups. It is also usual to acknowledge any financial support received in relation to this work.

# Contents

# List of Figures

# List of Tables

# Glossary

| | |
|---|---|
| $N$ | Number of residual blocks in a network |
| $F$ | Number of convolutional filters per layer |
| $d$ | Embedding dimension (Transformer models) |
| $h$ | Number of attention heads |
| $L$ | Number of Transformer encoder layers |
| $\pi(a \mid s)$ | Policy: probability of selecting move $a$ in board state $s$ |
| $v(s)$ | Value: predicted game outcome from board state $s$ |
| $\mathbf{X}$ | Node feature matrix in GNN input |
| $\mathbf{A}$ | Adjacency matrix representing piece connectivity or legal moves |
| cp | Centipawn; one hundredth of a pawn's value, used by engines to score positions |
| ELO | Rating system for measuring relative playing strength |
| ply | A half-move; one move by either White or Black |

# Abbreviations

| | |
|---|---|
| CNN | Convolutional Neural Network |
| CUDA | Compute Unified Device Architecture |
| DTZ | Distance to Zeroing (tablebase metric) |
| ELO | Elo rating system (named after Arpad Elo) |
| FEN | Forsyth–Edwards Notation |
| GAT | Graph Attention Network |
| GCN | Graph Convolutional Network |
| GNN | Graph Neural Network |
| MCTS | Monte Carlo Tree Search |
| MPS | Metal Performance Shaders (Apple GPU backend) |
| NNUE | Efficiently Updatable Neural Network |
| PGN | Portable Game Notation |
| PV | Principal Variation |
| SE | Squeeze-and-Excitation (block) |
| UCI | Universal Chess Interface |
| WDL | Win/Draw/Loss (evaluation format) |
| YAML | YAML Ain't Markup Language |

# Chapter 1

# Introduction

## 1.1 Problem Statement

Chess has been a benchmark for Artificial Intelligence research for decades. It is a controlled, fully observable environment that exposes how well a machine can reason under combinatorial pressure. Early milestones such as IBM's Deep Blue relied on handcrafted heuristics and "brute force" calculation. While effective, these systems were restrictive: they could not generalise beyond the specific rules their creators encoded.

Today, neural-heavy architectures like AlphaZero, MuZero, and Leela Chess Zero dominate competitive computer chess. These engines have pushed ELO ratings beyond the 3500 mark, effectively ending the era of human–computer competition. But this strength comes at a cost. Modern engines depend on massive computational overhead and deep search trees to maintain their advantage.

## 1.2 Motivation

Current state-of-the-art engines are strong, but they are also "computationally greedy." Their performance comes from exploring millions of potential future states (search) before selecting a move. This reliance on search is a barrier for mobile and edge-computing applications.

This project pursues "Search-less Chess." By training Convolutional Neural Networks (CNNs), Transformers, and Graph Neural Networks (GNNs), we investigate whether a model can develop a "grandmaster's intuition," the ability to look at a board and immediately identify the best move without a search algorithm. Removing search would reduce move latency and could reveal how deep learning architectures internalise spatial and strategic relationships.

## 1.3 The Challenge of Chess

Chess is a game of perfect information, yet it remains computationally "unsolved" because of its strategic depth. A player must balance tactical immediate threats (checks, captures) with abstract long-term goals (piece mobility, pawn structure, king safety).

### 1.3.1 Computational Complexity

The complexity of chess is often illustrated by Shannon's Number, which estimates the game tree complexity at approximately $10^{120}$. Even the state space, the number of possible legal positions, is estimated at $10^{40}$. Because the number of possible variations grows exponentially with every half-move (ply), brute-force calculation is physically impossible. This requires an approach that can either prune the search space or, as we propose, bypass it entirely through high-fidelity policy prediction.

### 1.3.2 Failure of Naive Approaches

Simple heuristics (e.g., assigning a point value to pieces) fail because chess is highly contextual; a Queen is worthless if the King is in an inescapable checkmate. Naive machine learning models often fall into "greedy" play, capturing material while ignoring a looming positional collapse. Without the "look-ahead" provided by search, a model must be considerably more sophisticated to recognise the subtle cues that separate a winning position from a losing one.

## 1.4 Critique of Existing Solutions

Engines like Stockfish 18 are strong but function as "black boxes" of search optimisation. They require significant CPU/GPU resources to reach their peak ELO. Even hybrid models like Leela Chess Zero, which use neural networks for evaluation, still rely on Monte Carlo Tree Search (MCTS) to validate their "hunches."

## 1.5 Overview of My Approach

### 1.5.1 Machine Learning Architectures

This project compares three architectures to determine which best captures the tactical and strategic patterns of chess without search.

- **CNNs:** Treat the board as a 2D image, capturing local spatial patterns and piece interactions.

- **Transformers:** Use self-attention mechanisms to model long-range dependencies across the board (e.g., a Bishop's influence from a8 to h1).
- **GNNs:** Represent the board as a dynamic graph where pieces are nodes and their legal moves are edges, emphasising the relational geometry of the game.

We train these models using the Lichess Stockfish dataset, effectively "distilling" the knowledge of a 3500-ELO engine into our search-less architectures.

### 1.5.2 Comparison Methodology

To validate our approach, we benchmark our models against Stockfish at varying "depth" constraints. This lets us simulate different ELO ranges and see where a search-less model begins to falter compared to a searching engine. We also use the Lichess Puzzle dataset to test the models on specific tactical motifs, giving a granular look at their "chess IQ" in areas like forks, pins, and endgame transitions.

## 1.6 Summary of Key Results

## 1.7 Scope and Limitations

The goal of this research is not to compete with the world's strongest engines, but to push the ceiling of what is possible within the "search-less" paradigm.

We acknowledge that without search, our models may struggle with "horizon effects," where a tactic requires deep calculation beyond the model's immediate pattern recognition. Highly technical endgames (e.g., Rook and Bishop vs. Rook) may also remain difficult for a purely intuitive model.

## 1.8 Aims and Objectives

The primary objective is to engineer and evaluate deep learning models (CNN, Transformer, GNN) capable of achieving a 2000 ELO rating without search algorithms. Success is measured by:

1. Inference speed: whether the models show a meaningful reduction in move generation time compared to traditional engines.
2. Generalisation: whether the models have "learned" chess principles rather than memorised positions, verified via performance on unseen puzzle sets.

3. Architectural comparison: which architecture (spatial, relational, or sequential) is best suited for the geometry of chess.

## 1.9 Report Structure

Chapter 2 surveys the history of chess AI and the development of deep learning in games. Chapter 3 details our methodology, covering data curation, normalisation, and the hyperparameters of each architecture. Chapter 4 presents the experimental results, including ELO benchmarking and tactical performance. Chapter 5 discusses the implications of these findings for lightweight, search-free AI.

# Chapter 2

# Literature Review

## 2.1 Classical Chess Engines

Early chess engines relied on the minimax algorithm paired with alpha-beta pruning to search the game tree efficiently.

These engines used handcrafted evaluation functions that scored board positions based on material count, piece activity, king safety, and other heuristics. Deep Blue is the canonical example.

While effective, these approaches were limited in adaptability and required extensive domain knowledge to tune. Stockfish, the strongest open-source chess engine, exemplifies this classical approach with its highly optimised search algorithms and evaluation function.

## 2.2 Probabilistic Search Methods

Monte Carlo Tree Search (MCTS) is a probabilistic search method that builds a search tree through random sampling of the game space, exploring promising moves without exhaustive search. In complex game states where the search space is vast, MCTS has clear advantages over traditional minimax. Leela Chess Zero, for instance, combines MCTS with neural network evaluations.

MCTS also adapts its search strategy based on prior simulation outcomes, making it more flexible in dynamic game environments. This flexibility extends beyond chess: MCTS is central to AlphaGo's Go play and to several general game-playing systems.

## 2.3 The Deep Learning Revolution

AlphaZero, developed by DeepMind, combined deep learning with self-play reinforcement learning and defeated Stockfish in a 2017 match. Its architecture has two components: a policy network that predicts move probabilities and a value network that estimates position outcomes. Through self-play alone, with no human game data, AlphaZero surpassed human-level play in chess, shogi, and Go.

Leela Chess Zero replicates AlphaZero's architecture and training methodology as an open-source project, and is now competitive with Stockfish at top-level play. The broader impact of AlphaZero's approach has been a shift toward neural network architectures and self-play training in chess engine research.

## 2.4 Supervised Learning Approaches

DeepChess uses a two-stage deep neural network: a Pos2Vec autoencoder pre-trained unsupervised on 640K human games from the CCRL database, followed by a supervised comparison network that takes two positions as input and outputs which is more favourable. The system never assigns numerical evaluations; it directly learns to rank positions, reaching grandmaster-level play with no hand-crafted heuristics.

Giraffe trains a neural network evaluation function using temporal-difference learning via self-play, with minimal hand-crafted input features. Despite receiving no explicit human chess knowledge, the resulting engine plays at approximately FIDE International Master level (top 2.2% of rated players), comparable to the strongest handcrafted engines of the time. The key difference from DeepChess is that Giraffe uses self-play rather than human game databases, and TD-learning rather than supervised pairwise classification.

## 2.5 Recent Innovations

Graph representations of chess positions attempt to capture the relational structure of the game more directly than grid-based encodings. Rigaux and Kashima (2024) proposed a graph-based neural network architecture that models interactions between pieces and their board positions, reporting improved evaluation accuracy and strategic understanding over grid baselines.

Explainable AI (XAI) in chess is a newer research direction. Svendsen and Tørresen (2022) developed methods to visualise the influence of individual pieces and moves on engine evaluations, making the decision-making process of chess engines more transparent to both players and researchers.

## 2.6 Game AI Frameworks

The python-chess library provides move generation and validation, PGN parsing and writing, Polyglot opening book reading, Syzygy/Gaviota tablebase probing, and full UCI/XBoard engine communication. It is the standard Python interface for working with chess engines programmatically.

## 2.7 Web-Based Chess Platforms

### 2.7.1 Existing Interactive Chess Platforms

Lichess, developed by Thibault Duplessis, is an open-source chess server written in Scala/Play with an asynchronous architecture using Akka actors, MongoDB for game storage (800M+ games), Elasticsearch indexing, and nginx proxying both HTTP and WebSocket connections. Real-time move streaming is handled via WebSockets.

### 2.7.2 Bot-vs-Bot Comparison Platforms

The Top Chess Engine Championship (TCEC) is the most prominent example of a spectator-oriented bot-arena platform. The Chessprogramming Wiki documents TCEC's format and various tournament manager implementations.

### 2.7.3 Human-vs-Bot Interfaces

The Lichess Board API endpoints (boardGameStream, boardGameMove) are the reference implementation for streaming live moves between a bot and a human player in real time, using REST with NDJSON streaming.

### 2.7.4 Web Technologies for Real-Time Games

The WebSocket Protocol (IETF RFC 6455, 2011) is the standard for full-duplex communication over a single TCP connection, and is the protocol underlying the game communication layer in this project.

The choice of Go and Svelte for the backend and frontend respectively is uncommon in game development but well-suited to this project. Go's concurrency model handles multiple simultaneous games and real-time interactions naturally, while Svelte's reactive framework provides efficient UI updates with minimal overhead.

## 2.8 Critical Analysis

Three architectural families dominate current chess AI research: CNNs, transformers, and graph neural networks. Each has distinct strengths and weaknesses that motivate the comparative approach taken in this project.

Krieg et al. (2024) confirm that CNNs remain competitive in hybrid NNUE + $\alpha$-$\beta$ deployments and are the most computationally efficient at inference time. However, CNNs have a structural limitation: a 3×3 kernel sees only adjacent squares, so many stacked layers are needed to propagate information across the board. This is a poor match for chess, where a single piece (e.g., a rook or queen) influences the entire board instantly.

Transformers address this receptive field problem through global self-attention. Chessformer (CF-240M) achieves 2347 Elo and 93.5% puzzle accuracy at 30× fewer FLOPs than comparable AlphaZero-scale models. The critical finding from Keller and Loepfe (2024) is that positional encoding is the decisive design choice: absolute, relative, and dynamic (Shaw et al.) encodings produce markedly different results. However, quadratic attention complexity over 64 tokens, while manageable, is costly at scale compared to CNN inference, and transformers require careful positional encoding since absolute embeddings fail to capture board geometry and naive relative encodings lose piece-type information.

Ruoss et al. (2024) train a large transformer on 10 million Lichess games annotated with Stockfish 16 at 50ms per position. This is architecturally the closest existing work to the pipeline in this project, covering action-value and state-value heads and the tradeoffs between square-based and piece-based tokenisation strategies. The critical gap is that Ruoss et al. evaluate a single architecture; this project systematically compares Square and Piece Transformers under identical conditions.

Graph neural networks offer a third alternative. AlphaGateau (GAT + GATEAU edge-feature layer) outperforms ResNet-based AlphaZero by an order of magnitude in sample efficiency on a 5×5 chess variant, showing that graph representations encode relational structure (legal moves as edges) that grids cannot express compactly. Yet Rigaux and Kashima train only on a 5×5 variant and fine-tune to 8×8; no published work presents a full GCN/GAT chess engine trained end-to-end on standard chess from scratch against CNN and Transformer baselines on the same dataset. This is the most direct gap this project fills.

## 2.9 Summary

Two findings from the literature directly shape the methodology of this project. First, representation matters more than depth. Chessformer shows that positional encoding is the decisive variable for transformers, while Rigaux and Kashima show that graph edges (encoding legal moves) capture relational structure that grid-based

representations miss. A multi-architecture comparison is the natural empirical test of this claim. Second, supervised Stockfish annotation is a viable and scalable training signal: Ruoss et al., Molinelli et al., and Oshri and Khandwala all validate this pipeline, but none investigate PV unrolling for position diversity.

The central gap in the existing literature is the absence of a fair cross-architecture benchmark. Every existing paper evaluates one architecture in isolation. This project's controlled comparison on identical Lichess/Stockfish data with PV-unrolled positions addresses that gap directly.

# Chapter 3

# Methodology and Design

## 3.1 Requirements Analysis

The framework must support a fair, controlled comparison of deep learning architectures for search-less chess. The following requirements guided its design.

### 3.1.1 Functional Requirements

1. The framework must support training and inference across at least three neural network families—CNNs, Transformers, and GNNs—under identical data and evaluation conditions.
2. Any backbone (e.g., ResNet, GAT) must be composable with any output head (Policy, Value, or Dual) without code duplication.
3. The system must ingest engine-annotated positions from Parquet files and train models using both hard and soft policy labels alongside centipawn-derived value targets.
4. A multi-stage data preparation pipeline must handle over one billion positions, including deduplication, PV line unrolling, tablebase enrichment, and offline pre-encoding into binary shards.
5. An automated benchmarking pipeline must pit trained models against Stockfish at configurable depths, recording per-move evaluations and exporting PGN files for post-hoc analysis.
6. A common agent interface must allow any model, engine, or random baseline to participate in games interchangeably.

### 3.1.2 Non-Functional Requirements

1. Training must use mixed-precision arithmetic (float16) and `torch.compile` where applicable to maximise GPU throughput.
2. The data pipeline must stream from disk one row-group at a time, bounding peak RAM usage regardless of dataset size.
3. Each subsystem (encoders, models, training loops, agents) must reside in its own package with clearly defined interfaces, so that it can be developed and tested independently.
4. All hyperparameters must be specified in a single YAML configuration file, and training checkpoints must be restorable for exact experiment replay.
5. Adding a new backbone or encoder should require implementing a single abstract class without modifying existing training or evaluation code.

## 3.2 Framework Architecture

The framework is organised into five principal modules, each responsible for a distinct concern.

### 3.2.1 Chess Environment Module

The chess environment module wraps the `python-chess` library to provide board management and move encoding. A `BoardWrapper` class exposes properties for legal moves, game-over detection, and piece queries. A statically generated lookup table maps every possible UCI move string (including promotions) to a unique integer index; this table contains 4544 entries and serves as the universal action space for all policy heads.

Three specialised state encoders translate a `chess.Board` object into the tensor format each architecture family expects:

- **CNNEncoder:** In its initial form, the encoder produces an $18 \times 8 \times 8$ tensor with 12 bitboard planes (six piece types per colour), four constant planes for castling rights, one plane for the en passant square, and one plane encoding the side to move as $+1$ (White) or $-1$ (Black). The board is always encoded from the current player's perspective—ranks and files are flipped when Black is to move. This encoder was later extended with tactical feature planes and material imbalance channels (Section 3.3.6).
- **GNNEncoder:** Constructs a graph with 64 nodes, one per square. Each node carries a 14-dimensional feature vector: a 13-element one-hot piece identifier and a colour scalar. Edges come from two sources—

physical adjacency (King move patterns) and tactical relations (`board.attackers`)—and carry three-dimensional attributes: normalised Manhattan distance, a binary ray-tracing flag for unblocked lines of influence, and a binary legality flag. Reverse edges are added to preserve structural symmetry while keeping directional legality features intact.

- **TransformerEncoder:** Supports two tokenisation schemes. The **SquareTokenizer** emits a fixed-length sequence of 64 tokens (one per square) with a 13-class vocabulary (empty plus twelve piece types), normalised rank/file coordinates, and full attention. The **PieceTokenizer** emits a variable-length sequence of up to 32 tokens (one per piece), ordered by colour (current player's pieces first), with each token carrying its piece type and square position. Padding masks prevent empty slots from contributing to attention.

All three encoders inherit from an abstract `StateEncoder` base class that mandates `encode()`, `get_input_shape()`, and `name` methods. The factory layer uses these to select the correct encoder at runtime based on the chosen backbone.

### 3.2.2 AI Agent Module

The agent module defines a common interface through the abstract `ChessAgent` class, which requires every agent to implement `get_move(board, time_limit)` and a `name` property. Four concrete agents are provided:

- **RandomAgent:** Selects uniformly at random from the legal move set and serves as the lower-bound baseline.

- **LearningAgent:** Wraps any trained `ChessModel` and its corresponding encoder. At inference time it encodes the board, runs a forward pass, masks illegal moves by setting their logits to $-\infty$, and picks the move via greedy argmax or temperature-controlled sampling with optional top-$k$ filtering.

- **MCTSAgent:** Implements AlphaZero-style Monte Carlo Tree Search. Each node stores visit counts, cumulative value, and a prior probability from the policy network. Selection follows the PUCT formula $\text{UCB} = Q + c_{\text{puct}} \cdot P \cdot \frac{\sqrt{N_{\text{parent}}}}{1+N}$. Leaf nodes are evaluated by the value head, and values are back-propagated with sign flips to account for alternating perspectives. Dirichlet noise ($\alpha = 0.3$, $\varepsilon = 0.25$) can be added at the root for exploration during self-play data generation.

- **UCIAgent:** Communicates with any UCI-compatible engine (e.g., Stockfish) via `python-chess`'s engine protocol. It supports configurable depth, time limits, skill levels, and MultiPV analysis for obtaining centipawn-scored move distributions.

### 3.2.3 Training and Evaluation Engine

The training subsystem centres on the `Trainer` class, which orchestrates the supervised training loop.

A loss factory selects the appropriate loss function based on the head type. `PolicyLoss` computes KL divergence against soft label distributions or cross-entropy against hard labels with optional label smoothing. `ValueLoss` uses mean squared error between the predicted tanh-bounded value and the target. `DualLoss` combines both with configurable weights. The trainer wraps the forward pass in `torch.autocast` (float16 on CUDA/MPS) and applies `GradScaler` on CUDA to avoid underflow in low-precision gradients. Non-GNN backbones are passed through `torch.compile` for graph-level kernel fusion and optimised execution.

Two learning rate schedules are supported—cosine annealing and step decay—both configured via YAML. Model weights, optimiser state, scheduler state, epoch counter, and best validation loss are serialised to disk after every epoch. Intermediate checkpoints are pruned to keep only every fifth epoch, the best, and the final model.

As training progressed, the initial streaming Parquet pipeline was superseded by a multi-stage offline data preparation system designed for over one billion positions. This pipeline is described in Section 3.3.9.

## 3.3 Design Decisions and Justification

### 3.3.1 Choice of Python and PyTorch

Python was selected for its dominance in the machine learning ecosystem and the maturity of supporting libraries (`python-chess`, `pyarrow`, `torch_geometric`). PyTorch was preferred over TensorFlow for its eager-mode debugging, first-class support for dynamic computation graphs (essential for the variable-length PieceTransformer and graph-structured GNN inputs), and its `torch.compile` JIT pathway for production-grade throughput.

### 3.3.2 Backbone–Head Decoupling

Backbone feature extraction is deliberately separated from task-specific output heads. Every backbone subclasses `ChessModel` and implements `forward_backbone()` to produce a fixed-dimension feature vector; the `set_head()` method then attaches one of three interchangeable heads. This decoupling yields $6 \times 3 = 18$ valid backbone–head combinations from six backbone and three head implementations, without combinatorial code explosion. More importantly, it means the only variable when comparing architectures is the backbone itself—the output layer, loss function, and training loop stay identical.

### 3.3.3 Streaming Data Pipeline

The dataset is stored in Apache Parquet format, which supports columnar compression and row-group-level random access. The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, reads one row-group at a time and shuffles within each group. Peak memory is therefore bounded to the size of a single row-group (typically 64–128 MB), regardless of total dataset size—critical when scaling to hundreds of millions of positions. Multi-worker data loading is supported by partitioning row-groups across workers using their `worker_id`.

### 3.3.4 Move Encoding

Rather than the $8 \times 8 \times 73$ spatial policy planes used by AlphaZero, a flat vector of 4544 logits is used. Each index corresponds to a unique UCI move string, generated exhaustively from all valid (from_square, to_square, promotion) tuples. This representation is architecture-agnostic: the same policy head attaches to CNNs, Transformers, and GNNs without modification. The trade-off is the loss of spatial locality in the policy output, which the flat head compensates for with a hidden layer.

### 3.3.5 Dual-Headed AlphaZero-Style Architecture

The default configuration uses a dual-headed model that jointly predicts the policy distribution and the board evaluation from shared backbone features. This follows the AlphaZero paradigm: the value head provides a training signal for the backbone to learn positional understanding, while the policy head learns tactical move selection. The combined loss $\mathcal{L} = \lambda_p \cdot \mathcal{L}_{\text{policy}} + \lambda_v \cdot \mathcal{L}_{\text{value}}$ is weighted equally by default ($\lambda_p = \lambda_v = 1.0$).

### 3.3.6 Enhanced Encoder Design

After initial experiments with the base 18-channel CNN encoder, the encoder was extended with explicit tactical feature planes and material imbalance channels. The goal was to inject domain knowledge that the model would otherwise need to discover from raw piece positions alone.

The following $8 \times 8$ tactical planes were added. Four attack/defence maps give per-square counts of attackers and defenders for each side, normalised by the maximum count; these make piece safety immediately visible to the first convolutional layer. Two mobility planes record the number of legal moves available to each piece, broadcast to its square, encoding piece activity without requiring the model to trace move generation logic. A single pin detection plane carries binary flags for pieces absolutely pinned to their King. Two King safety planes hold heuristic scores for each side's King based on pawn shield integrity and open files near the King.

Five material imbalance planes round out the extension. Rather than encoding material as a single scalar

difference, five constant $8 \times 8$ planes encode the per-piece-type count difference (our count minus their count) for Pawns, Knights, Bishops, Rooks, and Queens independently. This lets the model distinguish strategically different configurations—two Knights versus a Rook, for instance—that a scalar sum would conflate.

The extended encoder thus produces a $32 \times 8 \times 8$ tensor (18 base + 9 tactical + 5 material), with the additional channels computed directly from the `python-chess` board state.

### 3.3.7 Spatial Policy Head

The initial policy head applied global average pooling to the backbone's spatial feature maps before projecting to the 4544-move logit vector. This discards square-level information critical for move selection—which piece on which square should move where. The spatial policy head replaces global average pooling with a $1 \times 1$ convolution that reduces the channel dimension, followed by a flatten operation that preserves the $8 \times 8$ spatial grid. The resulting $8 \times 8 \times C'$ representation is then projected to move logits via a linear layer, letting the network associate specific board locations with specific moves.

### 3.3.8 Squeeze-and-Excitation Blocks

Squeeze-and-Excitation (SE) blocks were added to the ResNet backbone's residual blocks. Each SE block applies global average pooling to "squeeze" the spatial dimensions into a channel descriptor, passes it through a two-layer fully connected bottleneck with ReLU activation, and uses the resulting channel weights to "excite" (scale) the original feature maps via element-wise multiplication. This channel attention mechanism lets the ResNet dynamically prioritise different feature channels depending on the position—emphasising pawn structure planes in endgames, for example, and King safety planes in middlegames. The SE blocks sit inside the residual blocks so that skip connections maintain gradient stability.

### 3.3.9 Data Pipeline Evolution

The initial streaming Parquet pipeline worked well for prototyping but became a bottleneck at scale: runtime board encoding via `python-chess` was CPU-bound and could not saturate GPU training. A five-stage offline pipeline replaced it:

1. **Deduplication (DuckDB):** FEN strings are normalised (stripping move counters) and grouped using DuckDB's out-of-core SQL engine. When duplicate positions exist, the analysis with the highest depth or node count is retained.

2. **PV Unrolling:** For each base position, the first $N$ moves of the principal variation are played out. Each

intermediate position inherits the root evaluation (with sign flips for alternating sides), avoiding the cost of re-evaluating with Stockfish. Drift analysis confirmed that 96% of inherited evaluations remain within ±100 centipawns of the true value.

3. **Tablebase Enrichment:** Positions with five or fewer pieces are probed against Syzygy tablebases using `chess.syzygy.Tablebase`. Hits replace engine evaluations with exact WDL (Win/Draw/Loss) values and the DTZ-optimal move as the policy target, giving mathematically perfect labels for the endgame.

4. **Target Computation:** Engine centipawn scores are converted to sparse top-20 policy distributions (stored as `uint16` move indices and `float16` scores) and normalised value targets (`float16`).

5. **Binary Sharding:** Processed positions are written into shards of 100K–500K positions in a compact binary format using packed bitboards (12 `uint64` values + 2 metadata bytes per position, approximately 98 bytes each), which allows memory-mapped loading with zero deserialisation overhead.

A cleanup pass filters positions with analysis depth $\leq 10$ or zero node counts. Decisively won positions ($|\text{cp}| > 1500$) and trivial mate-in-1/2 positions are downsampled to prevent value head bias, while the ±100–500 centipawn range—where precise evaluation matters most—is upsampled.

The resulting dataset occupies approximately 180–200 GB for one billion positions. It loads via a `torch.utils.data.Dat` (map-style) with true random shuffling, `persistent_workers`, and configurable `prefetch_factor`.

## 3.4 Chess Implementation

Chess rules are not reimplemented; the framework delegates all legality checking, move generation, and game-over detection to the `python-chess` library. The `BoardWrapper` class is a thin convenience layer over `chess.Board`, exposing properties such as `legal_moves_uci`, `is_checkmate`, and `halfmove_clock`. This delegation keeps the codebase focused on machine learning concerns while relying on `python-chess` for correctness in edge cases like en passant, castling through check, under-promotion, and the fifty-move rule.

Board representation is determined by the encoder (Section 3.2.1) rather than by a single canonical format. Each model family therefore receives the board in its most natural form: a spatial tensor for CNNs, a graph for GNNs, and a token sequence for Transformers.

## 3.5 Web Platform Design

### 3.5.1 Platform Requirements

The web platform provides a browser-based interface for interacting with trained models. Its core requirements are:

- A user must be able to play a full game of chess against any trained model (Human-vs-Bot mode).
- A user must be able to select two models and watch them play against each other in real time (Bot-vs-Bot mode).
- The interface must support model selection from all available checkpoints, displaying metadata such as architecture type and parameter count.
- Move updates must be streamed in real time with sub-second latency.

### 3.5.2 System Architecture

The platform follows a client-server architecture. The backend is written in Go, chosen for its lightweight goroutine-based concurrency model, which maps naturally to managing multiple simultaneous games without thread-pool overhead. The frontend uses Svelte, a compile-time reactive framework that produces minimal JavaScript bundles and efficient DOM updates—well suited to the frequent, localised state changes involved in rendering a chess board.

The frontend and backend communicate over WebSockets (RFC 6455) for bidirectional, low-latency move streaming. A REST API handles session management, model listing, and game creation; the WebSocket channel carries move events and clock updates during active games.

### 3.5.3 Game Modes

The platform supports two game modes. In **Human-vs-Bot** mode, the user makes moves through the board UI; each move is sent to the backend, which invokes the selected model's `get_move` method and streams the response back. In **Bot-vs-Bot** mode, the backend orchestrates alternating `get_move` calls between two selected models, broadcasting each move to connected spectators with a configurable delay for readability.

### 3.5.4 User Interface Design

The interface centres on an interactive chessboard rendered using an SVG-based board component. Move input is handled via drag-and-drop with legal-move highlighting. A side panel displays the move list in standard algebraic notation, a real-time evaluation bar (when a value head is available), and model selection controls. The design prioritises clarity over decoration, following conventions established by Lichess.

### 3.5.5 Backend API Design

The backend exposes the following core endpoints:

- `GET /api/models` — List available model checkpoints with metadata.
- `POST /api/games` — Create a new game session, specifying mode and model(s).
- `WS /api/games/{id}/ws` — WebSocket endpoint for real-time move streaming.
- `POST /api/games/{id}/move` — Submit a human move (Human-vs-Bot mode).

The backend loads PyTorch models via a Python subprocess bridge, communicating board states and receiving moves over a local socket. This isolates the Python ML runtime from the Go HTTP server, preventing GIL contention from affecting WebSocket responsiveness.

## 3.6 Technical Challenges

Several technical challenges arose during design and were addressed through targeted engineering decisions.

GNN inputs cannot be trivially stacked along a batch dimension because each graph has a variable number of edges. The collation function addresses this by offsetting edge indices by $i \times 64$ for the $i$-th graph in the batch and concatenating all edges into a single $(2, \sum E_i)$ tensor, with a `batch` vector mapping each node to its graph. This follows the PyTorch Geometric batching convention.

The PieceTransformer presents a different batching problem: its sequences range from 1 to 32 tokens depending on the number of pieces on the board. A key-padding mask, constructed from the `attention_mask` field, prevents padded positions from influencing attention computation.

Mixed-precision training required platform-specific handling. The MPS backend supports float16 autocast but not `GradScaler`. The trainer therefore enables the scaler only on CUDA devices while still using autocast on MPS for throughput gains.

Finally, engine-annotated data provides centipawn scores for the top-$N$ moves, making soft policy labels possible via a softmax over normalised scores. These labels carry more information than a one-hot best-move label but require KL divergence rather than cross-entropy as the loss function. The pipeline supports both modes, selectable via configuration.

# Chapter 4

# Implementation

## 4.1 Development Environment and Tools

The framework is implemented in Python 3.10 with PyTorch as the deep learning backend. Table 4.1 summarises the principal libraries and their roles.

Table 4.1: Key libraries and their roles in the framework.

| **Library** | **Purpose** |
|---|---|
| PyTorch | Neural network definition, training, and inference |
| PyTorch Geometric | Graph convolution and attention layers (GCN, GAT) |
| python-chess | Board representation, move generation, engine protocol |
| PyArrow | Parquet file I/O for the training dataset |
| NumPy / Pandas | Numerical operations and data analysis |
| Matplotlib | Evaluation flow charts and training curve visualisation |
| PyYAML | Configuration file parsing |
| tqdm | Progress bars for training and data generation |

All experiments ran on two hardware configurations: an Apple M-series workstation using the MPS (Metal Performance Shaders) backend for local development, and an NVIDIA GPU cluster (CUDA) for large-scale training. The codebase auto-detects the best available device at startup—preferring CUDA over MPS over CPU—and enables hardware-specific optimisations (TF32 math and cuDNN auto-tuning on CUDA; float16 autocast on MPS).

Version control uses Git. All hyperparameters live in a single YAML configuration file (`config/default.yaml`), parsed at runtime into a hierarchy of Python dataclasses (`Config`, `ModelConfig`, `TrainingConfig`, etc.), so every experiment is fully reproducible from its configuration snapshot.

## 4.2 Core Framework Implementation

### 4.2.1 Abstract Base Classes

The framework's extensibility rests on two abstract base classes:

- `ChessModel(ABC, nn.Module)`: Every neural network backbone subclasses this and implements `forward_backbon` (returning a fixed-dimension feature vector) and `get_backbone_output_dim()`. The base class provides a concrete `forward(x)` that chains the backbone with whichever output head has been attached via `set_head()`.
- `StateEncoder(ABC)`: Every board encoder implements `encode(board)` and `get_input_shape()`. This contract lets the data pipeline and agents work with any encoder–model pairing without type-checking.

### 4.2.2 Factory Pattern

The `create_model(config)` factory reads the backbone name and head type from the configuration, instantiates the appropriate backbone with its architecture-specific parameters (e.g., number of residual blocks, number of attention heads), constructs the matching output head, and attaches it. A companion `get_encoder_for_model(backbone)` factory maps backbone names to their corresponding encoder classes. This two-step factory keeps backbone–encoder pairings consistent.

### 4.2.3 Configuration System

All tuneable parameters are centralised in a YAML file and parsed into nested dataclasses:

- `HardwareConfig`: Device selection and data-loader worker count.
- `ModelConfig`: Backbone type, head type, and per-family sub-configs (`CNNConfig`, `TransformerConfig`, `GNNConfig`).
- `TrainingConfig`: Batch size, learning rate, weight decay, epoch count, loss weights, and LR scheduler type.
- `EngineConfig`: Path to the Stockfish binary and default search parameters.

A recursive `_dict_to_dataclass()` helper converts arbitrarily nested YAML dictionaries into their corresponding dataclass instances, giving type safety and IDE auto-completion throughout the codebase.

## 4.3 Supervised Learning Model

### 4.3.1 Data Preparation

Training data comes from the Lichess Stockfish evaluation dataset, which provides millions of positions annotated with engine analysis. The data is stored in Apache Parquet format for efficient columnar compression and row-group-level random access.

The `ChessDataset` class, implemented as a PyTorch `IterableDataset`, supports two Parquet schemas:

1. **Engine-evaluated:** Each row contains a FEN string, the principal variation (PV) line in UCI notation, a centipawn evaluation, and an optional mate-in-$N$ score. The first move of the PV line serves as the best move.
2. **Legacy multi-move:** Each row contains a FEN, the best move, the game result, and up to 20 alternative moves with their centipawn scores. This schema enables soft policy labels.

Data loading works in four steps. First, the dataset enumerates all row-groups across all Parquet files. These row-groups are then partitioned across DataLoader workers by worker ID. Within each worker, row-groups and rows are shuffled independently. Finally, each row is converted to a training example by encoding the board and building the policy and value targets.

**Policy targets** are constructed in one of two modes. In *soft label* mode, the centipawn scores of the top-$N$ moves are normalised by their absolute maximum, passed through a softmax, and scattered into a sparse 4544-dimensional vector. In *hard label* mode, the best move receives a probability of 1.0.

**Value targets** are derived from the strongest available signal: the game result ($+1$, 0, $-1$), the mate score (mapped to $\pm 1$), or the centipawn evaluation (divided by 1000 and clamped to $[-1, 1]$). The value is always expressed from the current player's perspective.

A custom `collate_fn` batches examples into the format expected by each model family. CNN inputs are stacked along the batch dimension. Transformer inputs have each dictionary field (tokens, positions, masks) stacked independently. GNN inputs require more care: edge indices are concatenated with per-graph offsets of 64 nodes, and a `batch` vector maps each node to its originating graph, following the PyTorch Geometric batching convention.

### 4.3.2 Neural Network Architecture

Six backbone architectures are implemented, two per model family.

#### 4.3.2.1 Convolutional Neural Networks

**ConvNet** consists of six sequential convolutional blocks, each comprising a $3 \times 3$ convolution (with padding to preserve spatial dimensions), batch normalisation, and ReLU activation. The first block projects the 18-channel input to 256 channels; subsequent blocks maintain this width. A global average pooling layer reduces the $256 \times 8 \times 8$ feature maps to a 256-dimensional vector.

**ResNet** begins with an initial $3 \times 3$ convolution that projects the input to 256 channels, followed by $N$ residual blocks (configurable to 6, 10, or 20). Each residual block contains two $3 \times 3$ convolutions with batch normalisation, and the input is added to the output via a skip connection before the final ReLU. Global average pooling produces the output vector. The skip connections stabilise training in deeper configurations by providing gradient shortcuts.

#### 4.3.2.2 Transformers

**SquareTransformer** tokenises the board as 64 fixed tokens (one per square) using a vocabulary of 13 (empty + 12 piece types). Each token is embedded and summed with a learnable positional embedding and a coordinate projection (normalised rank and file). Side-to-move information is broadcast as a bias to all tokens, while castling rights are injected into a prepended CLS token. The sequence passes through $N$ Transformer encoder blocks, each using pre-LayerNorm, multi-head self-attention (8 heads), and a GELU-activated feed-forward network with a $4\times$ expansion ratio. The CLS token's final hidden state serves as the backbone output.

**PieceTransformer** tokenises the board as a variable-length sequence of up to 32 tokens (one per piece). Each token encodes the piece type (vocabulary of 12) and its square position via separate embedding tables, plus a sequence-order embedding. A key-padding mask prevents attention from flowing to padded positions. The architecture otherwise mirrors the SquareTransformer, with a CLS token aggregating the sequence representation.

#### 4.3.2.3 Graph Neural Networks

**GCN** projects the 14-dimensional node features to a hidden dimension of 256 via a linear layer, then applies $N$ GCN layers. Each layer performs a graph convolution (`GCNConv`), batch normalisation, and ReLU activation with a residual connection. Global mean pooling aggregates node embeddings into a single graph-level vector, which is summed with a side-to-move embedding and a castling projection before a final linear output layer.

**GAT** follows the same structure but replaces `GCNConv` with `GATConv`, which computes multi-head attention (4 heads) over neighbours. The GAT layers accept the 3-dimensional edge attributes (distance, ray-tracing,

legality) via the `edge_dim` parameter, allowing the attention mechanism to weight edges by their tactical significance. ELU activation replaces ReLU for smoother gradients in the attention computation.

### 4.3.3 Training Process

#### 4.3.3.1 Loss Functions

Three loss modules are provided:

- **PolicyLoss:** KL divergence for soft labels ($\mathcal{L}_{\text{KL}} = \sum_i t_i \log \frac{t_i}{p_i}$, where $t$ is the target distribution and $p = \text{softmax}(\text{logits})$) or cross-entropy for hard labels, with optional label smoothing.
- **ValueLoss:** Mean squared error between the predicted value (tanh-bounded) and the target.
- **DualLoss:** Weighted sum $\mathcal{L} = \lambda_p \mathcal{L}_{\text{policy}} + \lambda_v \mathcal{L}_{\text{value}}$.

#### 4.3.3.2 Optimisation

Training uses the AdamW optimiser with a default learning rate of $10^{-3}$ and weight decay of $10^{-4}$. Learning rate scheduling supports cosine annealing (default) and step decay. Gradients are clipped to a maximum norm of 1.0 to prevent instability during early training.

#### 4.3.3.3 Mixed Precision and Compilation

On CUDA devices, training uses float16 autocast with `GradScaler` to maintain numerical stability. On MPS (Apple Silicon), autocast is enabled but the scaler is disabled—MPS does not support it. Non-GNN backbones are wrapped with `torch.compile` for kernel fusion and operator-level optimisation. I found that GNN models had to be excluded from compilation because their dynamic graph structures prevent effective tracing.

#### 4.3.3.4 Checkpointing

After each epoch, the trainer serialises the model state dict, optimiser state, scheduler state, epoch number, global step count, and best validation loss. At the end of training, intermediate checkpoints are pruned to keep only every fifth epoch, reducing storage overhead while preserving enough granularity for post-hoc analysis.

## 4.4 Architectural Enhancements

After the initial implementation and early training runs, several architectural improvements were made to address weaknesses I observed.

### 4.4.1 Enhanced CNN Encoder

The base 18-channel encoder was extended to 32 channels. The 14 new planes are computed from the `python-chess` board state at encoding time:

- **Attack/Defence Maps (4 planes):** Per-square attacker and defender counts for each side, computed using `board.attackers()` and normalised by their board-wide maximum.
- **Mobility (2 planes):** For each piece, the number of legal moves is counted and broadcast to its square. One plane per side.
- **Pin Detection (1 plane):** Absolute pins to the King are detected by temporarily removing each friendly piece and checking whether the King becomes attacked along the vacated ray.
- **King Safety (2 planes):** A heuristic score per side based on pawn shield integrity (presence of pawns on the second and third ranks adjacent to the King) and open files near the King.
- **Material Imbalance (5 planes):** Per-piece-type count differences (Pawn, Knight, Bishop, Rook, Queen), broadcast as constant $8 \times 8$ planes.

### 4.4.2 Spatial Policy Head

The initial policy head used global average pooling, collapsing the $C \times 8 \times 8$ feature maps into a $C$-dimensional vector before projecting to the 4544 move logits. This discards spatial locality. The spatial policy head replaces GAP with a $1 \times 1$ convolution that reduces the channel count to $C'$, followed by `flatten()`, producing a $C' \times 8 \times 8 = 64C'$ vector. A final linear layer maps this to 4544 logits. This turned out to be important: it lets the network associate specific board squares with specific move origins and destinations.

### 4.4.3 Squeeze-and-Excitation Blocks

SE blocks were inserted into each `ResidualBlock`. After the second convolution and batch normalisation, the feature maps are squeezed via global average pooling to a channel vector of dimension $C$, passed through a bottleneck (`Linear(`$C$`,` $C/r$`)` $\rightarrow$ ReLU $\rightarrow$ `Linear(`$C/r$`,` $C$`)` $\rightarrow$ Sigmoid, with reduction ratio $r = 16$), and the

resulting channel weights multiply the original feature maps. The SE block sits inside the residual path, so the skip connection ensures stable gradient flow even in deep (20-block) configurations.

### 4.4.4 Flash Attention

The Transformer encoder blocks were updated to use `torch.nn.functional.scaled_dot_product_attention` in place of the explicit QKV matmul and softmax. On supported hardware, this dispatches to the Flash Attention kernel, reducing attention memory from $O(n^2)$ to $O(n)$ and giving a measurable training speedup for the 64-token SquareTransformer sequence length.

### 4.4.5 Activation Functions

ReLU activations in the ConvNet and ResNet backbones were replaced with SiLU (Swish), which provides smoother gradients near zero. The Transformer blocks kept GELU from the initial implementation.

## 4.5 Data Pipeline Evolution

The initial streaming Parquet pipeline (Section 4.5.1) worked well for prototyping but became the training bottleneck at scale—per-row `python-chess` encoding could not keep up. A five-stage offline pipeline was built to support over one billion positions.

### 4.5.1 Stage 1: Deduplication

DuckDB's out-of-core SQL engine normalises FEN strings (stripping halfmove and fullmove counters), groups by the normalised FEN, and retains only the row with the highest analysis depth or node count. This eliminates redundant positions that would otherwise bias the training distribution toward common openings.

### 4.5.2 Stage 2: PV Line Unrolling

For each base position, the first $N$ moves of the principal variation are played out using `python-chess`. Each intermediate position is emitted as a separate training example, with the subsequent PV move as its policy target. The root evaluation is inherited with sign flips for alternating sides, avoiding the prohibitive cost of re-evaluating every unrolled position with Stockfish. A drift analysis validated this approximation: 96% of inherited evaluations remained within $\pm 100$ centipawns of the independently computed value.

### 4.5.3 Stage 3: Tablebase Enrichment

Positions with five or fewer pieces are probed against the Syzygy 5-piece tablebases (approximately 1 GB on disk) using `chess.syzygy.Tablebase`. On a hit, the engine's approximate evaluation is replaced with the mathematically exact WDL outcome (+1, 0, or −1), and the policy target is set to the DTZ-optimal (Distance To Zeroing) move. At inference time, the agent bypasses the neural network entirely for $\leq$ 5-piece positions, playing the tablebase-optimal move directly.

### 4.5.4 Stage 4: Target Computation and Cleanup

Engine centipawn scores are converted to sparse policy distributions: only the top 20 moves are stored as pairs of `uint16` indices and `float16` scores (81 bytes per position). Value targets are stored as `float16` (2 bytes).

Several dataset cleanup rules apply:

- Positions with analysis depth $\leq 10$ or zero node counts are removed.
- Decisively won positions ($|\text{cp}| > 1500$) and trivial mate-in-1/2 are downsampled to prevent value head bias toward extreme evaluations.
- The $\pm$100–500 centipawn range, where precise evaluation matters most for playing strength, is upsampled.

### 4.5.5 Stage 5: Binary Sharding

Processed positions are packed into binary shards of 100K–500K positions. Each position is stored as 12 `uint64` bitboards plus 2 metadata bytes (approximately 98 bytes for the board state), alongside the sparse policy and value targets. The resulting dataset occupies roughly 180–200 GB for one billion positions and is loaded via a map-style `torch.utils.data.Dataset` with memory-mapped access, true random shuffling via `DistributedSampler`, `persistent_workers=True`, and configurable `prefetch_factor`.

## 4.6 Baseline Models

Two baselines anchor the evaluation:

- **RandomAgent:** Plays uniformly at random from the legal move set. This provides a lower bound on playing strength and verifies that trained models have learned meaningful patterns.
- **UCIAgent (Stockfish):** Wraps Stockfish at configurable depth and skill levels. At full strength (depth

15+), Stockfish serves as the upper bound. At reduced depths (1–5), it provides intermediate calibration points for estimating ELO.

## 4.7 Benchmarking and Evaluation Pipeline

The benchmarking system automates model evaluation through six stages:

1. Each trained model plays a configurable number of games against Stockfish, alternating colours for fairness. Both raw-policy (greedy argmax) and MCTS-augmented agents are supported.
2. After every move, Stockfish analyses the resulting position at depth 18, recording the centipawn score from White's perspective. Mate scores are converted to large constants ($\pm 10000$).
3. Each game is exported in Portable Game Notation with embedded evaluation comments, enabling review in standard chess software (Lichess, Chess.com, ChessBase).
4. Matplotlib generates evaluation flow charts for each game, plotting centipawn score against move number with win/draw/loss shading. A combined comparison plot overlays all models.
5. Per-move inference times are recorded for both the model and the Stockfish opponent, enabling latency comparisons.
6. A Markdown report is generated with win/draw/loss tables, timing statistics (mean, median, min, max), and links to PGN and figure files.

## 4.8 Testing and Debugging

Unit tests cover critical components: move encoding (verifying that all 4544 UCI moves round-trip through the index lookup), encoder output shapes, model forward-pass dimensions, and loss function gradients. Integration tests verify end-to-end training on a small synthetic dataset to catch dimension mismatches across the encoder–model–head pipeline.

Debug utilities include the `CNNEncoder.decode_piece_planes()` method, which reconstructs piece positions from an encoded tensor for visual inspection, and the `LearningAgent.get_move_distribution()` method, which returns the top-$k$ moves with their softmax probabilities for diagnosing policy quality.

## 4.9 Documentation

The codebase follows a consistent documentation standard: every module, class, and public method includes a docstring specifying its purpose, arguments, and return values. A `README.md` provides installation instructions, a quick-start guide, and example training commands. The `setup.sh` script automates environment creation and dependency installation.

# Chapter 5

# Evaluation

## 5.1 Evaluation Methodology

## 5.2 Supervised Learning Model Results

### 5.2.1 Training Performance

### 5.2.2 Playing Strength

### 5.2.3 Error Analysis

## 5.3 Comparative Analysis

## 5.4 Ablation Studies

## 5.5 Web Platform Evaluation

### 5.5.1 Usability Testing

### 5.5.2 Performance and Responsiveness

### 5.5.3 Bot-vs-Bot Match Results via Platform

## 5.6 Comparison with Related Work

## 5.7 Interpretation and Significance



## 5.8 Limitations and Challenges

# Chapter 6

# Critical Review and Reflection

## 6.1 Achievement of Objectives

The primary objective of this project was to engineer and evaluate a suite of deep learning models capable of playing chess without search algorithms. The framework implements all six proposed architectures (ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, GAT), each trainable under identical data conditions with interchangeable output heads. The modular backbone–head design and the streaming data pipeline both exceed the original specification, which anticipated only three architectures.

The secondary objectives of inference speed measurement and cross-architecture comparison are supported by the benchmarking pipeline, which records per-move timing and exports PGN files for qualitative analysis.

Beyond the initial architecture, the framework evolved during development. The CNN encoder was extended from 18 to 32 channels with tactical feature planes (attack/defence maps, mobility, pins, king safety) and per-piece-type material imbalance encoding. The ResNet backbone was augmented with Squeeze-and-Excitation blocks for dynamic channel attention. A spatial policy head replaced global average pooling to preserve square-level information. The Transformer models adopted Flash Attention via `scaled_dot_product_attention`. The data pipeline was also redesigned from a streaming Parquet reader to a five-stage offline system (DuckDB deduplication, PV line unrolling, Syzygy tablebase enrichment, target computation, and binary sharding) capable of handling over one billion positions.

## 6.2 What Worked Well

The abstract `ChessModel` interface made adding new architectures straightforward. Adding the GAT architecture, for instance, required implementing only a single class with two methods; the training loop, loss functions,

and evaluation pipeline required zero modifications. This backbone–head decoupling justified the upfront design effort many times over.

The row-group-level streaming from Parquet files allowed training on datasets larger than available RAM without any changes to the training loop. This mattered most when scaling from initial prototyping (100K positions) to full-scale runs (tens of millions of positions). Centralising all hyperparameters in a single YAML file eliminated hard-coded constants and made experiment management straightforward. Combined with checkpoint serialisation, any past experiment can be exactly reproduced.

Using softmax distributions over the engine's top-$N$ move evaluations rather than one-hot best-move labels provided richer gradient information, especially for positions where multiple moves are nearly equally strong. The decision to inherit root evaluations (with sign flips) for PV-unrolled positions rather than re-evaluating each with Stockfish was both practical and accurate. The drift analysis showing 96% of inherited values within $\pm$100cp validated this approximation and enabled a large increase in training data diversity at negligible computational cost.

Augmenting the CNN encoder with explicit attack/defence maps, mobility, and pin detection planes gave the model domain knowledge that would otherwise require many convolutional layers to infer. This had the most visible effect on the shallower ConvNet, where the limited receptive field makes it difficult to learn long-range piece interactions from raw bitboards alone. Finally, using mathematically perfect endgame labels from Syzygy tablebases for $\leq$ 5-piece positions eliminated engine approximation error in the training signal for the endgame regime, and the inference-time bypass ensured flawless endgame play without any neural network involvement.

## 6.3 What Didn't Work Well

The GNN encoder is much slower than the CNN encoder because it must compute attack and defence relations for every square on every board. This makes the GNN data pipeline the bottleneck during training, despite the model itself being no larger than the CNN.

The original policy head used global average pooling, discarding spatial information critical for move prediction. This was identified early and replaced with a spatial policy head, but the initial training runs on the pooled representation produced noticeably weaker policy accuracy, confirming that square-level information is essential for move selection.

## 6.4 Lessons Learned

Filtering out low-depth evaluations and balancing the value distribution had a larger impact on model performance than simply increasing dataset size. Data quality consistently mattered more than data quantity.

The choice of board representation (spatial tensor vs. graph vs. token sequence) determines what information is accessible to the model and what must be learned from scratch. Encoder design turned out to be as consequential as model architecture.

The ConvNet, despite being the simplest model, served as an invaluable debugging tool. Once the ConvNet was training correctly, issues in more complex architectures (Transformer attention masks, GNN edge batching) could be diagnosed by comparison. Starting with the simplest architecture saved a lot of time.

Mixed-precision training requires platform-specific handling. The asymmetry between CUDA and MPS support for `GradScaler` required conditional logic that was non-obvious and initially caused silent numerical instability.

## 6.5 Personal Reflection

This project deepened my understanding of deep learning engineering beyond what I learned in lectures. Implementing graph neural networks from scratch, including the non-trivial batching of variable-edge graphs, required engaging with PyTorch Geometric at a level far beyond its tutorial examples. The Transformer implementations reinforced the importance of positional encoding design: absolute, relative, and learnable encodings produced noticeably different training dynamics on the same data.

Managing a multi-architecture codebase also developed my software engineering discipline. The abstract base class pattern, which initially felt like over-engineering for a single-developer project, turned out to be essential when I added the sixth architecture three months into development.

## 6.6 Future Work

### 6.6.1 Reinforcement Learning

The framework currently relies entirely on supervised learning from engine-annotated data. Two reinforcement learning paradigms are natural extensions. The MCTSAgent already implemented in the framework could be used to generate self-play training data following the AlphaZero paradigm, recording MCTS visit-count distributions as policy targets and game outcomes as value targets, enabling the model to improve beyond the ceiling of its training data. Alternatively, playing the model against Stockfish at progressively increasing

difficulty levels (varying skill level and search depth) would provide a structured reinforcement signal. A curriculum from beginner (skill 0, depth 1) to club level (skill 5, depth 5) would allow the model to learn incrementally from opponents matched to its current strength.

### 6.6.2 Web Platform

The planned Go/Svelte web platform would provide a browser-based interface for interacting with trained models. Users could play against any trained model via a drag-and-drop board interface with real-time move streaming over WebSockets. Two models could play each other with spectator support, configurable move delay, and live evaluation bars. A round-robin tournament system where all trained models play against each other, with automated Elo calculation from the results, would allow systematic strength comparison. An analysis board mode for setting up arbitrary positions and comparing move recommendations of different models side by side is also planned.

### 6.6.3 Long-term Research Directions

Framing chess as a sequence prediction task, analogous to language modelling, where the model autoregressively predicts move continuations from a history of board states is one direction I find interesting. This Decision Transformer-style approach could enable genuine multi-move planning without explicit search. The backbone–head architecture is also game-agnostic, so adapting the framework to other perfect-information games (e.g., Shogi, Othello) would test the generality of the design. Extending the tablebase integration to 6- and 7-piece counts (at the cost of much more storage) would expand the range of positions with perfect play.

## 6.7 Broader Impact

The framework provides an open-source, controlled benchmark for comparing neural network architectures on a shared chess dataset. To the best of my knowledge, no such resource exists in the published literature. The modular design lowers the barrier to entry for researchers who wish to experiment with novel architectures or training regimes without reimplementing the full chess AI stack. The web platform, once complete, would provide a demonstration tool for communicating deep learning concepts to non-specialist audiences.

# Chapter 7

# Legal, Social, Ethical, and Professional Issues

## 7.1 Ethical Considerations

### 7.1.1 Research Ethics

This project does not involve human participants, personal data collection, or experimentation on living subjects. Ethical approval was obtained through the University of Surrey's SAGE (Self-Assessment for Governance and Ethics) process, which confirmed that the project falls within the low-risk category. All training data comes from publicly available chess game databases and open-source engine evaluations.

### 7.1.2 Responsible AI

The models developed here are narrow AI systems designed exclusively for chess move prediction and position evaluation, so they pose no direct risk of harm. That said, transparency in AI decision-making still matters. The framework's evaluation pipeline (centipawn tracking, PGN export) provides full traceability of every move decision, and the soft policy labels expose the model's uncertainty across candidate moves. Most commercial chess engines return only their top move without a probability distribution, so this level of interpretability is unusually high for the domain.

Bias in training data is worth noting: the Lichess Stockfish evaluation dataset reflects the engine's playing style and evaluation heuristics. Models trained on it will inherit Stockfish's strategic preferences — for instance, its material-focused evaluation — which may not represent alternative playing styles well. This is an acceptable limitation given that the project's goal is knowledge distillation from a strong engine, not modelling diverse human play.

## 7.2 Legal Issues

### 7.2.1 Intellectual Property

The framework is original work for the University of Surrey Final Year Project and complies with the University's intellectual property policy. No proprietary code or datasets are used.

### 7.2.2 Copyright and Licensing

All third-party libraries are used in compliance with their respective open-source licences:

- `python-chess`: GPL-3.0 — permits use, modification, and distribution.
- PyTorch: BSD-3-Clause — permissive licence with minimal restrictions.
- PyTorch Geometric: MIT — permissive licence.
- Stockfish: GPL-3.0 — used as an external binary for evaluation; the framework does not link against or modify its source code.
- Lichess game data: Creative Commons CC0 — public domain, no restrictions on use.

The training dataset comes from Lichess's publicly available database, released under Creative Commons CC0 (public domain). The Stockfish evaluations annotating this data are produced by an open-source engine and carry no additional copyright restrictions.

## 7.3 Social Issues

### 7.3.1 Accessibility

The framework is implemented in Python and uses well-documented libraries, so the barrier to entry is relatively low for anyone with basic Python experience. The YAML configuration system and command-line training scripts help with this. The main accessibility limitation is hardware: training requires a GPU for practical turnaround times, though the framework does run correctly on CPU.

### 7.3.2 Web Platform Accessibility

The planned web platform targets modern evergreen browsers (Chrome, Firefox, Safari, Edge) and is designed to be responsive across desktop and tablet screen sizes. The SVG-based board rendering ensures crisp display

at any resolution. Keyboard accessibility for move input and ARIA labels for screen readers are planned but not yet implemented.

### 7.3.3 Impact on Chess Community

The project contributes to the growing body of open-source chess AI research. By providing a controlled multi-architecture comparison and an accessible web interface, it could serve as an educational resource for players curious about how neural networks evaluate positions. The models are not strong enough to be useful for cheating in competitive play, and the framework includes no mechanisms for real-time assistance during online games.

## 7.4 Professional Issues

### 7.4.1 Code of Conduct

Development follows the BCS (British Computer Society) Code of Conduct. The framework is designed as an open educational and research tool, which satisfies the public interest obligation. The codebase follows standard software engineering practices — version control, automated testing, documentation, and modular design — in line with the professional competence requirement. All results, including negative results and known limitations, are reported honestly throughout this dissertation.

### 7.4.2 Information Security

The framework processes no personal data and stores no user credentials. On the web platform side, the backend API validates all inputs (FEN strings, move UCI strings) before processing, and no sensitive information such as API keys or passwords is hard-coded in the repository.

### 7.4.3 Web Platform Security

The planned web platform addresses common web vulnerabilities in a few ways. All FEN strings and UCI moves received from clients are validated against `python-chess` before execution, preventing malformed input from reaching the model. WebSocket connections use WSS (WebSocket Secure) in production deployments. Move submission endpoints are rate-limited to prevent denial-of-service via rapid move flooding.

## 7.5 Sustainability

### 7.5.1 Environmental Impact

Neural network training is computationally intensive and carries a real carbon footprint. This project takes several steps to reduce that cost. Mixed-precision training halves the memory bandwidth and increases throughput by up to 2×, cutting the wall-clock time and energy consumption of each training run. The streaming data loader avoids redundant I/O by reading each row-group exactly once per epoch, and checkpoint pruning reduces storage overhead by retaining only every fifth epoch.

More fundamentally, search-less inference is the whole point of the project: a single forward pass consumes orders of magnitude less energy than the millions of node evaluations a searching engine performs. If the models were ever deployed at scale, this difference would matter considerably.

### 7.5.2 UN Sustainable Development Goals

The SDGs most relevant to this project are SDG 4 (Quality Education), SDG 9 (Industry, Innovation, and Infrastructure), and SDG 12 (Responsible Consumption and Production). The connection to education is the most direct: the web platform is designed to let players explore how different neural architectures evaluate the same position, which makes deep learning concepts tangible in a way that reading about tensor shapes does not. The cross-architecture comparison — CNN, Transformer, and GNN on identical data — is a contribution to AI research methodology (SDG 9), though a modest one; the real value is the controlled experimental setup rather than any single architectural breakthrough. The efficiency measures described above (mixed precision, search-less inference) are a small but genuine effort toward responsible resource use (SDG 12), particularly given how energy-intensive large-scale model training has become across the field.

## 7.6 Summary

This project uses open-source tools and public-domain data throughout, produces transparent outputs with full move traceability, processes no personal information, and takes concrete steps to reduce its computational footprint through mixed-precision training and search-less inference. The main ethical consideration — bias inherited from Stockfish's evaluation style — is documented and acceptable given the project's distillation goals. No significant legal, social, or ethical risks were identified.

# Chapter 8

# Conclusion

This project investigated whether deep learning architectures can develop meaningful chess-playing ability without search algorithms. To this end, a modular framework was engineered that supports six neural network backbones (ConvNet, ResNet, SquareTransformer, PieceTransformer, GCN, and GAT), each trainable under identical conditions with interchangeable policy, value, and dual output heads.

The framework makes several contributions. First, it provides a controlled, cross-architecture comparison of CNN, Transformer, and GNN models trained on the same Lichess Stockfish evaluation data with consistent preprocessing, loss functions, and evaluation protocols. Second, it demonstrates that knowledge distillation from a strong engine via soft policy labels is a viable and scalable training signal for search-less models.

The project evolved beyond its initial design. The CNN encoder was extended from 18 to 32 channels with tactical feature planes (attack/defence maps, mobility, pin detection, king safety) and per-piece-type material imbalance encoding, injecting domain knowledge that would otherwise require many layers to learn. The ResNet backbone was augmented with Squeeze-and-Excitation blocks for dynamic channel attention, and a spatial policy head replaced global average pooling to preserve square-level information critical for move selection. The Transformer models adopted Flash Attention for improved memory efficiency and training throughput.

The data pipeline was redesigned from a streaming Parquet reader into a five-stage offline system capable of handling over one billion positions. DuckDB-based deduplication removes redundant positions, PV line unrolling generates diverse intermediate positions with inherited evaluations (validated at 96% accuracy within ±100cp), and Syzygy tablebase enrichment provides mathematically perfect labels for endgame positions with five or fewer pieces. The resulting binary-sharded dataset, with packed bitboard storage and sparse policy representations, enables efficient memory-mapped training at scale.

From a software engineering perspective, the backbone–head decoupling pattern, the factory-based model creation, and the YAML-driven configuration system collectively lower the barrier to experimentation: adding a

new architecture requires implementing a single abstract class, while all training, evaluation, and benchmarking infrastructure is reused without modification.

The project has several limitations. Without search, the models are expected to struggle with deep tactical sequences (the "horizon effect") and highly technical endgames beyond the tablebase's piece-count threshold. The GNN encoder's computational cost, driven by per-position attack relation computation, remains a training throughput bottleneck despite the offline pre-encoding pipeline addressing this for CNN and Transformer models.

Looking ahead, reinforcement learning via MCTS self-play and curriculum-based Stockfish RL is the clearest next step for pushing model strength beyond the supervised learning ceiling. The planned Go/Svelte web platform would provide an interface for human-vs-bot and bot-vs-bot interaction, and sequence modelling approaches (Decision Transformer-style autoregressive move prediction) could enable multi-move planning without explicit search.

In summary, this project shows that search-less chess is technically feasible and provides a useful testbed for comparing how different neural network families handle spatial, relational, and sequential structure. The framework, with its enhanced encoders, attention mechanisms, and billion-position data pipeline, provides the infrastructure for continued investigation.

# Appendix A

# Code Listings

# Appendix B

# Additional Results

# Appendix C

# User Guide

# Appendix D

# Game Examples

# Appendix E

# Ethical Approval Documentation

# Appendix F

# Project Planning Documents

# Bibliography


[1] J. A. Briffa, H. G. Schaathun, and S. Wesemeyer, "An improved decoding algorithm for the Davey-MacKay construction," in *Proc. IEEE Intern. Conf. on Commun.*, Cape Town, South Africa, May 23–27, 2010.

[2] A. A. el Gamal, L. A. Hemachandra, I. Shperling, and V. K. Wei, "Using simulated annealing to design good codes," *IEEE Transactions on Information Theory*, vol. 33, no. 1, pp. 116–123, Jan. 1987.

[3] P. G. Farrell, "Notes on source coding," Feb. 1992, university of Manchester.

[4] B. P. Tunstall, "Synthesis of noiseless compression codes," Ph.D. dissertation, Georgia Institute of Technology, 1968.

[5] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes in C: The Art of Scientific Computing*, 2nd ed. Cambridge Universty Press, 1992.

[6] B. Henderson, *Netpbm*, Feb. 22nd, 2009. [Online]. Available: http://netpbm.sourceforge.net/doc/

[7] *NVIDIA CUDA Programming Guide*, NVIDIA Corporation, Jul. 1st, 2009, version 2.3.

[8] M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," Digital SRC Research Report, Tech. Rep., 1994.